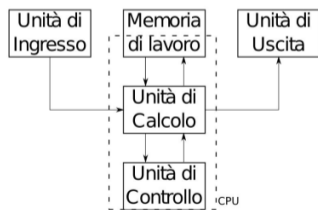


Gerarchie memoria

venerdì 27 febbraio 2026 13:11

- Andiamo a vedere ora come si ha l'interazione con la memoria . Per memoria si intende un dispositivo (esterno al processore) che permette di memorizzare informazioni . **Per memoria si intende generalmente la memoria primaria/di lavoro secondo architettura Von Neumann.**

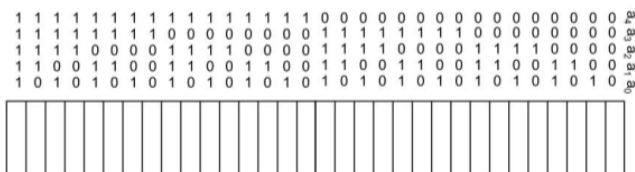


Quindi la memoria , generalmente viene suddivisa in 3 macro categorie :

1. Interna
 - a. Sono quelle memorie che vanno a lavorare sui registri
 - b. Lettura/scrittura dati da/verso processore
2. Principale
 - a. DRAM
 - b. Memorizza istruzioni e dati dei nostri programmi
 - c. Più lenta della memoria interna (3 a 1 colpi di clock)
3. Secondaria
 - a. Memoria di massa
 - b. Dimensioni della memoria più grande, ma più lenta

Quindi in virtù delle tecnologie , si cerca di arrivare ad un trade-off di : memorie costruite con transistor (aumenta il costo per bit immagazzinato) , ma diminuisce la velocità di accesso/latenza. Quindi costi vs tempo accesso.

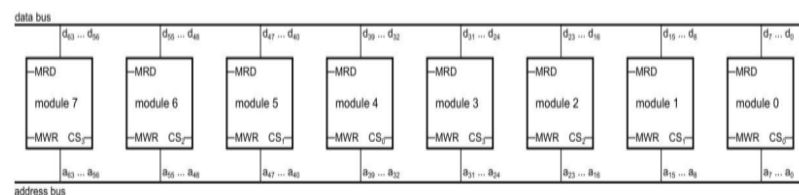
Andiamo a vedere ora su quella principale (modello flat):



Ma da notare che l'organizzazione logica della memoria non corrisponde ad organizzazione dei banchi di ram : non è facile accedervi. Quindi vista questa difficoltà, andiamo ad organizzare la ram in **banchi di memoria** : possibilità di letture in parallelo



In generale , si potrebbe essere interessati a porzioni di dati (8,16,32,64) byte di dati , quindi bisogna prevedere un meccanismo che faccia ciò :



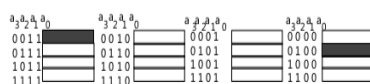
Abilitando solo il numero opportuno disegni CS(chip select): questo segnale (**aggiuntivo**) viene generato dal processore e mandato sul bus dati/indirizzo/control , in modo che il processore legga n moduli (n moduli con $\log_2 n$ per il numero di bit). Questo segnale rappresenta la taglia dell'operando in memoria con il quale abbiamo a che fare (8,16,32,64 -> byte , word , long word , quadword).

Sembrerebbe funzionare , ma in realtà non si è tenuto conto del disallineamento dei byte : **padding** :



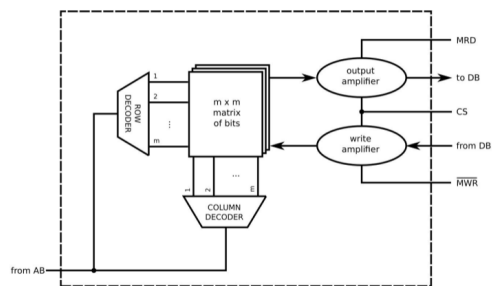
Quattro byte allineati sullo stesso indirizzo di riga.

Quattro byte disallineati.

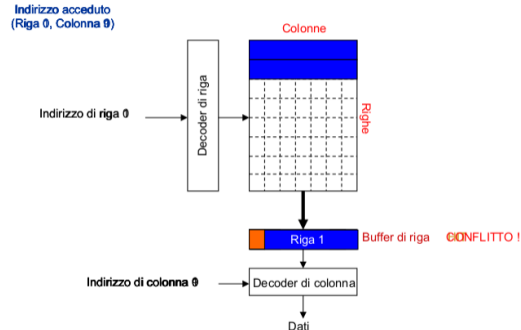


Due byte disallineati

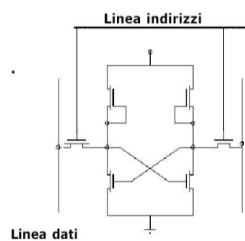
Notiamo che al massimo il disallineamento massimo è di 2 byte. Quindi in generale la memoria viene organizzata secondo matrici , nelle quali l'indirizzo di memoria che forniamo viene utilizzato per individuare una riga ed una colonna :



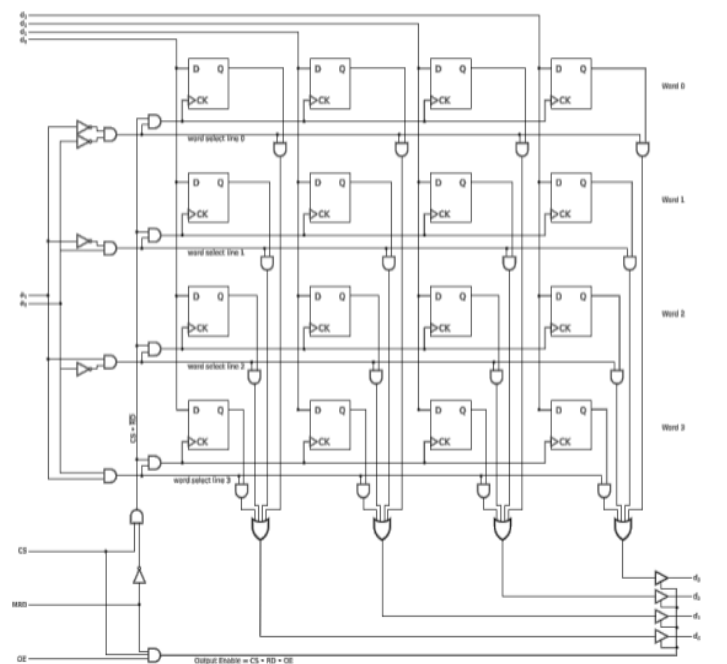
Attenzione però al ritardo di propagazione . La quale circuiteria è la seguente :



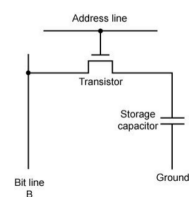
Tornando ai tempi di propagazione : le memorie per ovviare a ciò si usano di buffer tamponi (buffer riga), il quale contiene i dati che poi verranno amplificati. In dettaglio le ram sono fatte così : un flip flop fatto da 6 transistor (aumenta le prestazioni , ma aumentano i costi):



In dettaglio :

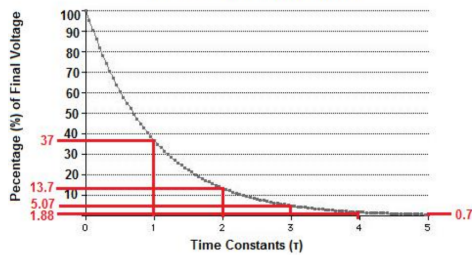


Tutto bello fino a qui , ma visto il costo e le attuali capacità della memoria questa soluzione non è accettabile : costerebbe assai realizzarla, quindi basandosi su quanto detto fino ad ora : si usano le **ram dinamiche**: utilizza un condensatore ed un transistor (carico vale 1 , scarico vale 0) , il quale funziona da interruttore.

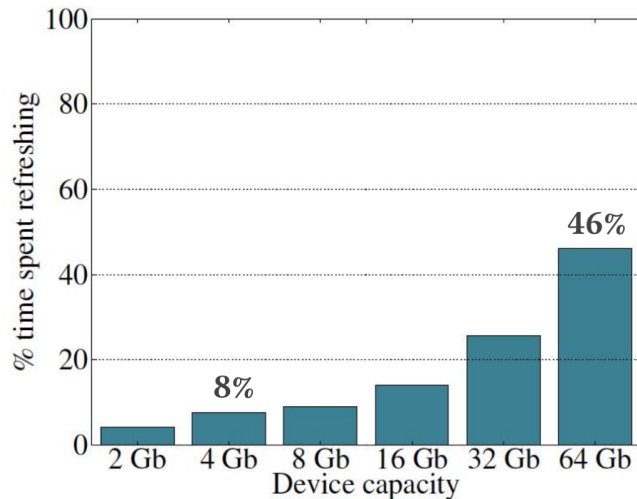


Quindi se il condensatore è carico si ha lettura/scrittura : se si ha tensione alta (1) su linea del bit si carica il condensatore e si scrive il dato , mentre per la lettura si seleziona la linea di indirizzo che si vuole , la carica del condensatore viene trasferita (attraverso amplificatore) al bit di linea ed infine si

ripristina la carica del condensatore. Ricordiamo che il tempo di scarica/carica del condensatore è molto veloce, facendo sì che il condensatore si scarichi fino al 63 % della sua carica iniziale (**costante temporale**). Quindi dopo questo tempo l'informazione è persa.

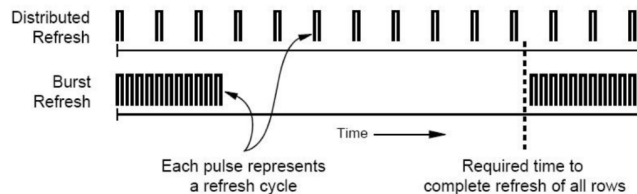


Quindi il controllore della memoria, deve rinfrescare tutte le righe per ripristinare la carica, il quale refresh viene fatto ogni 64 ms. Con questo refresh, sorge un problema: vi è sia un consumo energetico (ogni refresh consuma energia), sia degradazione delle prestazioni (durante il refresh la riga non è disponibile). Quindi riassumendo: durante il refresh vi possono essere lunghe pause e le dimensioni e la velocità sono limitate dalla necessità di refresh. In dettaglio:

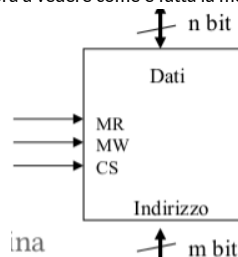


Quindi in dettaglio ci sono 2 tipi di refresh:

1. A burst
 - a. Ogni 64 ms fa il refresh delle righe
2. Distribuito
 - a. Le righe sono soggette a refresh in istanti temporali diversi
 - b. Questo refresh elimina le lunghe pause

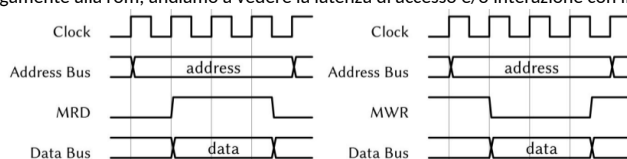


Andiamo ora a vedere come è fatta la memoria: è fatta come blocchetti:



Il quale è caratterizzato da indirizzo della parola da leggere/scrivere, MR (memory read) che se affermato permette di leggere, WR (write to register) che se affermato permette di scrivere nel registro, CS (chip select) che permette di abilitare uno o più moduli e l'insieme dei dati da processare.

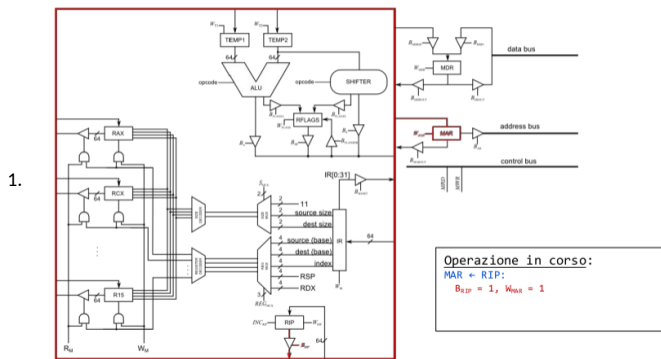
Analogamente alla rom, andiamo a vedere la latenza di accesso e/o interazione con il dispositivo:



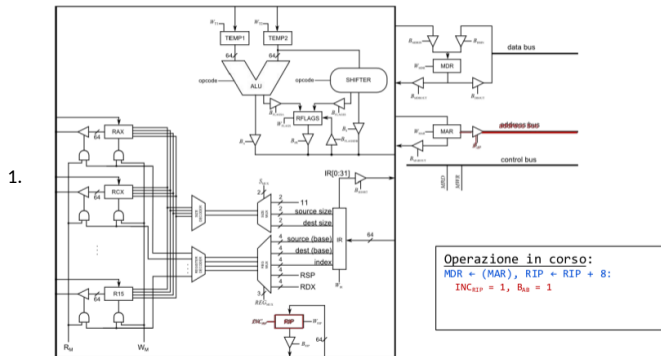
Quindi facciamo un esempio: voglio leggere/scrivere un indirizzo, quindi il dato contenente l'indirizzo viene mantenuto in memoria(buffer) con clock altro per tutto il tempo di interazione, per poi comunicare con il colpo clock successivo se sto leggendo e/o scrivendo. Quindi ora dopo la stabilizzazione, si riesce a leggere/scrivere da/verso memoria. Quindi vediamo ora cosa succede durante

la latenza di accesso , ovvero la fase di fetch : ricordiamoci che per fare il fetch di una qualunque istruzione servono 3 cicli di clock . Vediamolo in dettaglio :

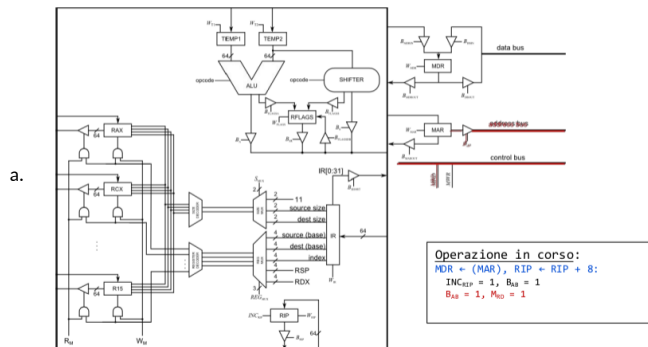
1. Fase 1



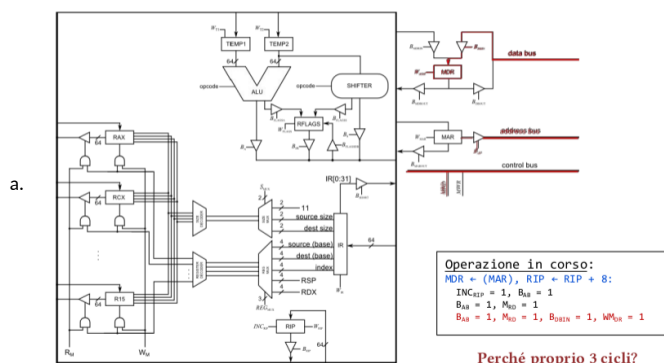
2. Fase 2:



3. Fase 3



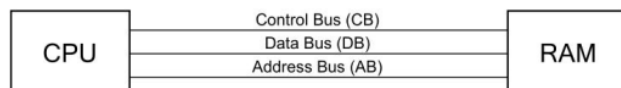
4. Fase 4:



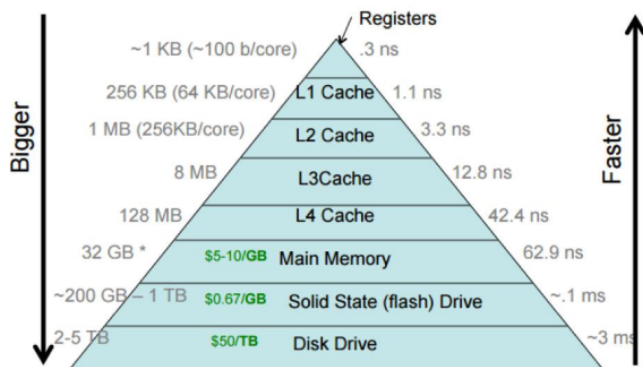
Quindi ora vista l'incompatibilità di alcune memorie , con alcuni processori , andiamo a vedere i parametri utili per vedere o meno questa compatibilità :

- CAS (column address strobe): ritardo (colpi di clock) tra istruzione/comando di read ed il momento in cui sono disponibili
 - Espresso in ciclo di clock per dram sincrone
 - Espresso in nanosecondi per dram asincrone
- Ritardo tra indirizzo riga ed indirizzo di colonna : numero cicli di clock per accedere ad una riga (copiato nel buffer tampone) e ne legge le colonne. Se questo tempo viene sommato al cas si ha il tempo necessario a leggere il primo bit dal bus dati
- Tempo di precaricamento di riga : tempo necessario per risolvere un conflitto se si è nella fase di caricamento di una riga
- Tempo di attivazione di riga : cattura il tempo di refresh.

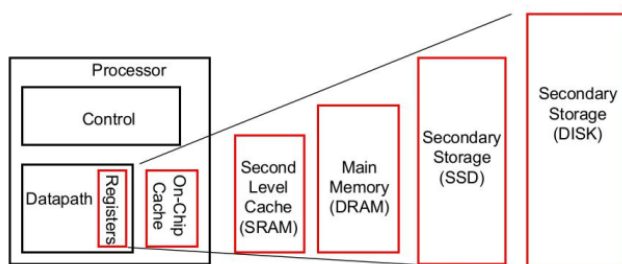
Ricordiamo che secondo il modello di Von-Neumann la CPU (processore) è direttamente collegata alla memoria (Ram) , ma avendo due velocità **completamente differenti**, si crea il famoso collo di bottiglia (la velocità di trasferimento è limitata dal dispositivo più lento) :



Che nonostante la *legge di Moore* non si riesce a trovare un compromesso buono . In virtù di quanto detto finora , è possibile avere una memoria a costo minimo e capacità massima e latenza accesso minima : si parla di **gerarchia di memoria** : ovvero di più livelli di memoria invisibili al programmatore :



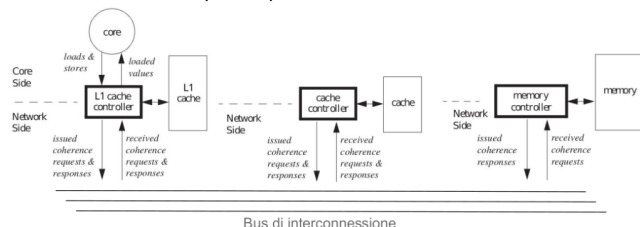
Da notare che sopra la memoria principale, ci sono delle memorie cache (registri tra memoria e processore) , aumentando così le dimensioni e smorzando il trade off. Vista la gerarchia di memoria, i dati vengono passati solo tra livelli adiacenti . Per ogni cache (in particolare per la cache L1) , si hanno due ulteriori suddivisioni : cache dati e cache istruzioni; Architettura Harvard. Quindi in generale (o meglio realmente) la gerarchia di memoria è realizzata nel seguente modo :



Basandosi quindi su prove empiriche , gli ingegneri usando dei pattern , sono arrivati a vedere il **principio di località** : ovvero uso della memoria con mascheramento della latenza in base al modo in cui è stato scritto il programma . Vediamolo in dettaglio : se in un qualunque istante , un programma tende ad accedere ad una porzione di dati relativamente piccola , sia per quanto riguarda i dati che le istruzioni : esecuzione locale del programma . **Si divide nel principio di località temporale e spaziale: il primo dice che se accedo ad un elemento nella memoria, quello stesso elemento potrà essere acceduto entro breve termine (nascondendo la latenza) , mentre la seconda dice se accedo ad un elemento, entro breve tempo accederò ad elemento "vicino"** . Per sfruttare al massimo il principio nel primo caso si tengono i blocchi acceduti più frequentemente vicino al processore, mentre nel secondo spostati i blocchi contigui tra livelli della gerarchia. Andiamo ora a vedere delle definizioni molto importanti :

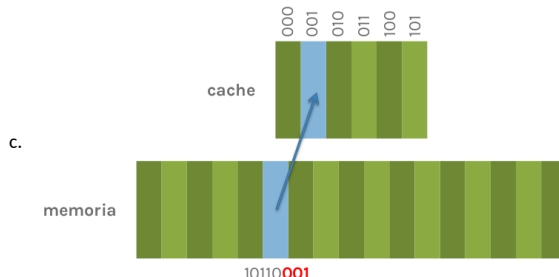
1. Hit rate : da massimizzare
 - a. Dati trovati in un livello vicino al processore , quindi programma più veloce e diminuisce la latenza
2. Miss rate : opposto dell'hit rate
 - a. Frequenza fallimento
 - b. (1-hit rate) : se scende l'hit rate sale
 - c. Se questo parametro è basso , di conseguenza altro è altro, si ha un utilizzo massimale del processore.
3. Miss penalty
 - a. Tempo necessario per trasferimento del blocco dal livello inferiore
4. Miss time : tempo fallimento
 - a. Tempo per ottenere il dato
 - b. Miss time= miss penalty + hit rate

Quindi si ha una situazione di questo tipo:

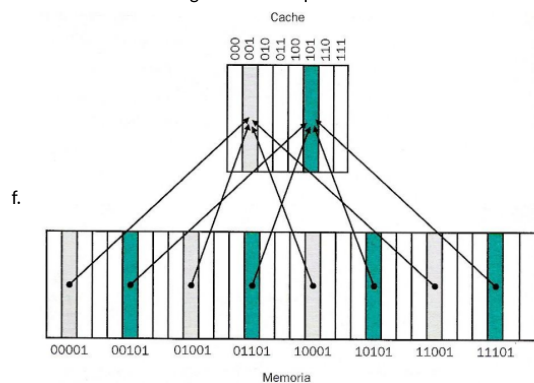


Dove il core (processore) parla solo con le cache di livello 1, passando per il controllore della cache di primo livello, il quale va a vedere se ci sono i dati nella cache di livello 1: se si sono ok, altrimenti li deve andare a "pescare" dal bus di interconnessione (collegato agli altri controllori). Andiamo ora a vedere in dettaglio come funziona questo collegamento: ogni cache è strutturata in linee, ognuna delle quali è formata da un blocco di 64 byte, ovvero la minima unità di dati che vengono passati tra le cache. Quindi per quanto detto prima, il **processore parla con il controllore della cache di primo livello**. Quindi se il dato è presente nella cache L1 si ha un hit! (finisce qui), altrimenti il controllore chiede agli altri controllori (chiede info al controllore di L2) e se non c'è neanche qui, si procede al livello 3 (controllore L3). Analogamente si fa quando si ha accesso in RAM. Tornando alle cache: sono di due tipi: **inclusive e non inclusive**: quella di livello superiore (più vicina al processore) mantiene informazioni riguardo i livelli inferiori, i quali memorizzano le info richieste. Quindi nel nostro caso solo le L1 vengono accedute direttamente dal processore. Mentre per le seconde: può capitare che non tutti i livelli siano inclusivi, come per esempio se miss in L1, si accede diretto in memoria, scrivendo così solo in L1, e se non vi fosse spazio, questo blocco scende in L2. In generale in una qualunque riga della cache, oltre ai dati veri e propri vi si scrive anche altri dati: le informazioni di controllo: come per esempio la directory. In virtù di queste informazioni aggiuntive, la dimensione della linea di cache è maggiore di 64 byte. Con queste ulteriori informazioni, si hanno politiche di accesso alle cache più efficienti. Vediamo ora le strategie di posizionamento/utilizzo cache:

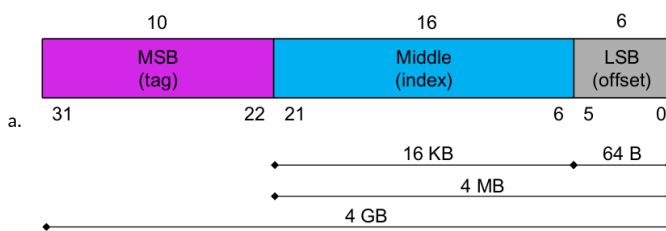
1. Accesso diretto: max hit rate non sapendo working set a priori
 - a. Uso la funzione hash per determinare il numero del blocco dove scrivere i dati
 - b. Funziona per via dell'indirizzamento a blocchi di 2^n



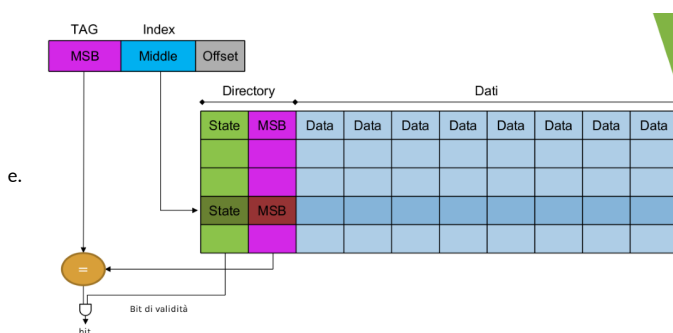
- d. Ma attenzione: ad una stessa posizione in cache, ci possono essere più parole (sinonimi):
- e. Si deve usare un tag per risolvere queste collisioni:



2. Accesso diretto: indirizzo memoria logico viene scomposto in 3 pezzi

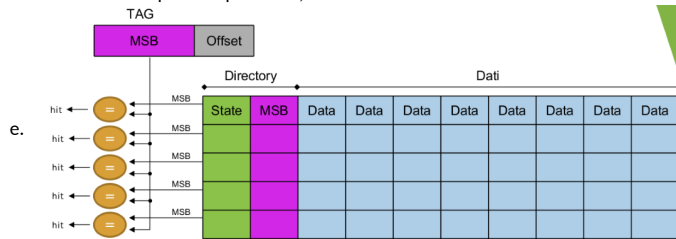


- b. Tag serve per discriminare sinonimi
- c. Index per selezionare una linea di cache
- d. Offset seleziona la parola corrispondente all'interno del blocco

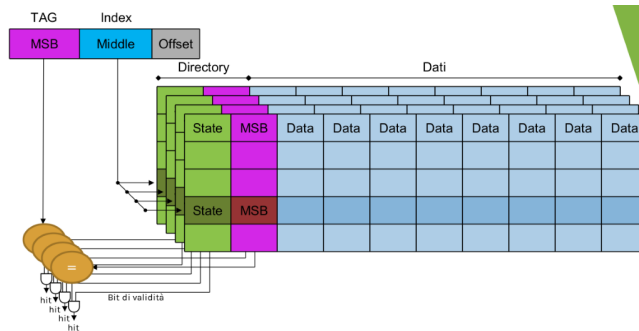


- f. Problema: miss sempre su stesso blocco, sotto utilizzo delle linee di cache

3. Cache completamente associativa : ogni blocco può essere inserito in una qualunque riga della cache , se in caso di miss
 - a. Quindi devo leggere il tag di tutte le righe
 - b. Utilizzo massimo cache
 - c. Riduzione conflitti
 - d. Contro:
 - i. Per ricerca devo scorrere tutta la cache
 - ii. Ricerca sequenziale troppo lunga : aumenta latenza
 - iii. Ricerca quindi in parallelo , aumentando così il costo



4. Cache set associativa a n-vie : cache che identifica e lavora su un insieme di linee , utilizzato secondo strategia completamente associativa



- a. Utilizzo migliore ma aumenta costo
- b. Ogni blocco può essere memorizzato in un insieme (≥ 2) di linee
- c. Ricerca su massimo di n tag
- d. Quindi il costo è accettabile
- e. Ed è attualmente la tecnologia utilizzata dai processori

Soffermiamoci sulle cache completamente associative e set associative a n-vie : si ha bisogno di scegliere una vittima per "rimpiazzare il blocco" , ma come viene scelta?

1. **Least recently used** : si ha una sorta di contatore dell'età
 - a. Il contatore mantiene il tempo finché non si ha il caricamento del blocco che viene azzerato , altrimenti se viene acceduto per vedere se libero si incrementa di uno
2. **Fifo (first in first out)** : Come LRU ma incremento avviene solo in caso di rimpiazzo
3. **Least frequently used** : difficile da implementare
4. **Round robin** (carosello): usato nelle completamente associative
 - a. Mantiene un puntatore alla linea da rimpiazzare , ma viene aggiornato ad ogni rimpiazzo
5. **Random** : come round robin , ma il puntatore viene aggiornato ad ogni accesso

Quanto detto finora , riguarda solo la lettura , ma andiamo ora a vedere la scrittura : ricordiamoci che il processore comunica solo ed esclusivamente con la cache di primo livello (L1) , la quale mantiene una copia dei dati . Quindi in virtù delle copie , se si è in presenza di aggiornamenti , si deve mantenere la consistenza con le copie , implicando prima il caricamento del dato e poi la copia dello stesso. Questa scrittura avviene secondo due modi : **Write through e Write back** : nella prima (scrivi attraverso) , se si scrive in L1 automaticamente si aggiornano le copie , quindi molto facile come implementazione ma prestazioni minori; mentre la seconda opta per aggiornamento delle copie (livelli sottostanti) se e solo c'è un rimpiazzo: quindi più difficile da implementare , ma prestazioni nettamente superiori. Andiamo ora a vedere le prestazioni invece : in generale una qualunque esecuzione di un programma impiega :

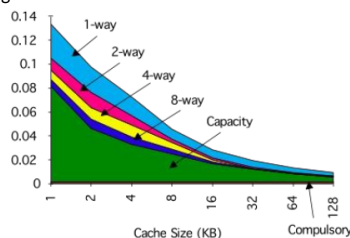
$$T_{exec} = n_{istruzioni} \cdot CPI \cdot \frac{sec}{ciclo}$$

Il numero di cicli di clock per istruzione (CPI) dipende dal Tempo

Medio di Accesso in Memoria (TMAM):

$$TMAM = Hit\ Time + (Miss\ Rate \cdot Miss\ Penalty)$$

Il quale se si riesce a ridurre, implica un aumento delle prestazioni. Torniamo ai miss, o meglio alle cause del miss : possibile **miss per conflitto, capacità ed obbligatori**. Vediamo il primo : si è nel caso si cache set associative a n-vie con blocchi di m byte , ma dato che $n < m$ si ha il conflitto. Come risolvere ? O aumentiamo il numero della dimensione della linea , oppure aumento la dimensione del blocco. In dettaglio :



Vediamo ora quelli obbligatori : la prima volta che si accede a quel dato non sarà disponibile (per esempio accendo il pc) , quindi si adotta la tecnica del cache prefetching : prevedo/indovino i blocchi che verranno richiesti in futuro , mentre nell'ultima si ha che la linea di cache , non può mantenere tutti i blocchi richiesti , quindi in seguito ad aumento della dimensione della cache aumentano i costi. Quindi in generale si ha una sorta di andamento a "vasca da bagno" :

